SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

1. *Introduction:*

FinSight AI is an AI-assisted financial health advisor designed for Malaysian SME owners who manage their financial documents manually without a dedicated finance team. This System Analysis Documentation introduces the technical solution behind FinSight AI, outlining how the system transforms uploaded financial documents such as bank statements, invoices, and receipts into clear, ringgit-based decisions through a combination of deterministic rule engines, an 8-week cash flow forecasting model, and Z AI GLM as an external LLM reasoning service.

1.1. *Purpose:*

This System Analysis Document defines the technical scope and design decisions behind the development of FinSight AI. It serves as the authoritative reference for the development team, QA team, and all project stakeholders throughout the UMHackathon 2026 sprint.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
   FinSight AI is a Flutter web application backed by a Node.js/Express server. The backend is organised into five feature-scoped service modules, all sharing a single MongoDB Atlas database accessed through a centralised Repository class. Binary file uploads are stored in Cloudflare R2, and AI reasoning is delegated to Z AI GLM as an external LLM-as-a-Service via a POST /v1/generated HTTP API.
2. ***Data Flows:***
   This document describes how financial data flows from the SME owner's browser through the Flutter frontend, the Node.js API layer, OCR parsing, MongoDB Atlas persistence, and Z AI GLM inference and returns as structured dashboard insights including risk alerts, cash flow forecasts, and ranked recommendations.
3. ***Model Process:***
   The document covers the end-to-end workflow of all five core features: Document Ingestion and Parsing, Cash Flow Forecast, Risk Detection and Alerts, AI Decision Recommendations, and the Financial Dashboard. Each feature's process is described from user trigger to final rendered output.
4. ***Role of Reference:***
   This document is the single technical reference for all developers and testers on the ChatStack Engineers team. It establishes module boundaries, API contracts, data schemas, and the LLM integration pipeline so that parallel development across the five features remains consistent and traceable throughout the 10-day sprint.

1.2. ***Background**:*

Malaysian SME owners across sectors such as F&B, automotive, and personal care typically manage their financial records manually across physical receipts, WhatsApp images, and spreadsheets. Without a dedicated finance team, these owners have limited visibility into their real-time cash flow position, overdue invoices, and rising expenses. Insights are usually derived from monthly bookkeeping rather than continuous monitoring, meaning cash flow problems are often identified only after they have already affected the bank balance.

FinSight AI was conceived to solve this problem by converting the documents SME owners already produce such as bank statements, supplier invoices, and receipts into actionable, ringgit-quantified financial intelligence using a local Malaysian large language model (Ilmu AI / Z AI GLM) as the reasoning engine.

1. *Changes in Major Architectural Components*
   As this is the initial version, all architectural components such as Flutter frontend, Node.js backend, MongoDB Atlas database, Cloudflare R2 file storage, and Z AI GLM integration are being introduced for the first time. The key architectural decision differentiating FinSight AI from a simpler CRUD application is the introduction of the PromptBuilder/ContextManager/TokenManager pipeline, which transforms raw financial records into structured LLM prompts while enforcing a hard token ceiling of 8,000 per call.

2. *New capabilities that are only introduced in this*
   The following capabilities are being introduced in version 1.0 of FinSight AI: automated document ingestion and OCR parsing for PDF, JPG, and PNG financial documents up to 15 MB; a deterministic five-rule risk detection engine producing severity-scored alerts with MYR-quantified impact; an 8-week cash flow forecasting model derived from historical transaction patterns; Z AI GLM-powered ranked action recommendations across four categories (Collections, Supplier Management, Cost Optimisation, and Financing) and a unified Financial Dashboard aggregating all five feature outputs into a single-screen view with a persistent critical alert banner and last-updated timestamp.

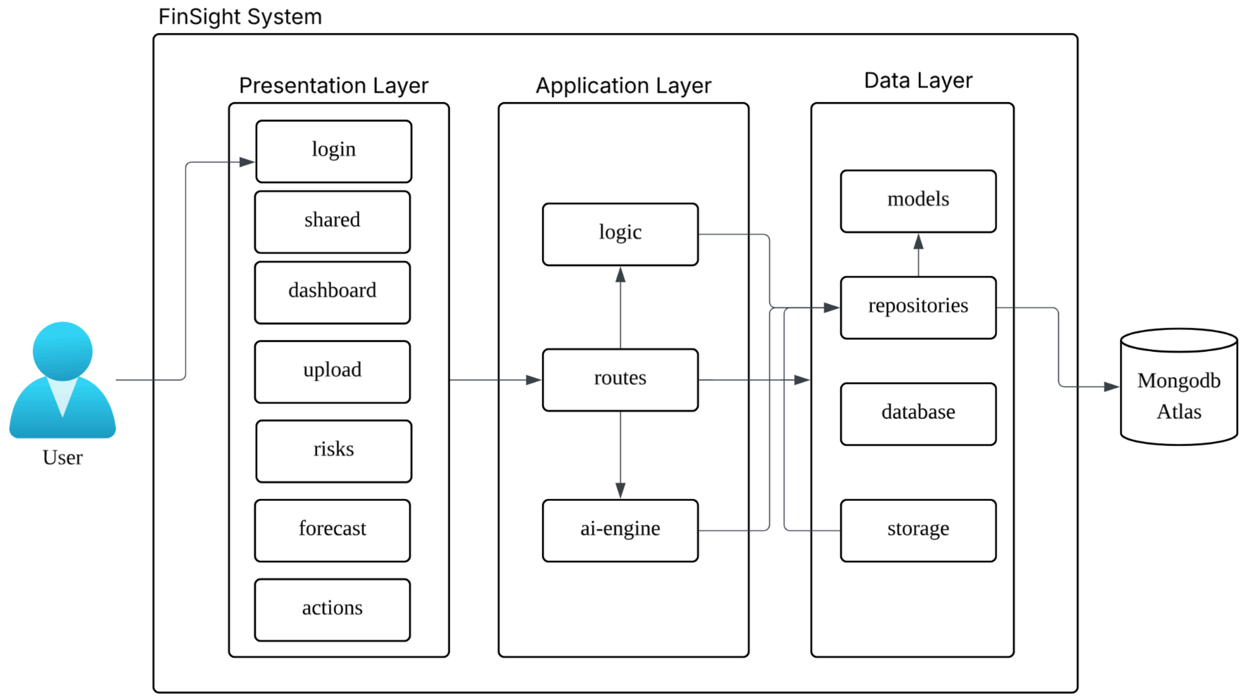
1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
SME Owner (primary user)	*Uploads documents, reviews dashboard, reads risk alerts, acts on AI recommendations*	*Usable without accounting knowledge; plain-English explanations with ringgit amounts; one screen shows everything important*
Development Team	*Builds and maintains the Flutter frontend, Node.js backend, and Ilmu AI integration*	*Clear module boundaries (five features); consistent API contract ({success, data,*

		error}); repository layer as the only database gatekeeper
QA Team	*Validates end-to-end flows across the five feature modules*	*Defined happy-path and negative-path cases per feature; graceful fallback when the system is unavailable*

2. System Architecture & Design

2.1. High Level Architecture



2.1.1. Overview:

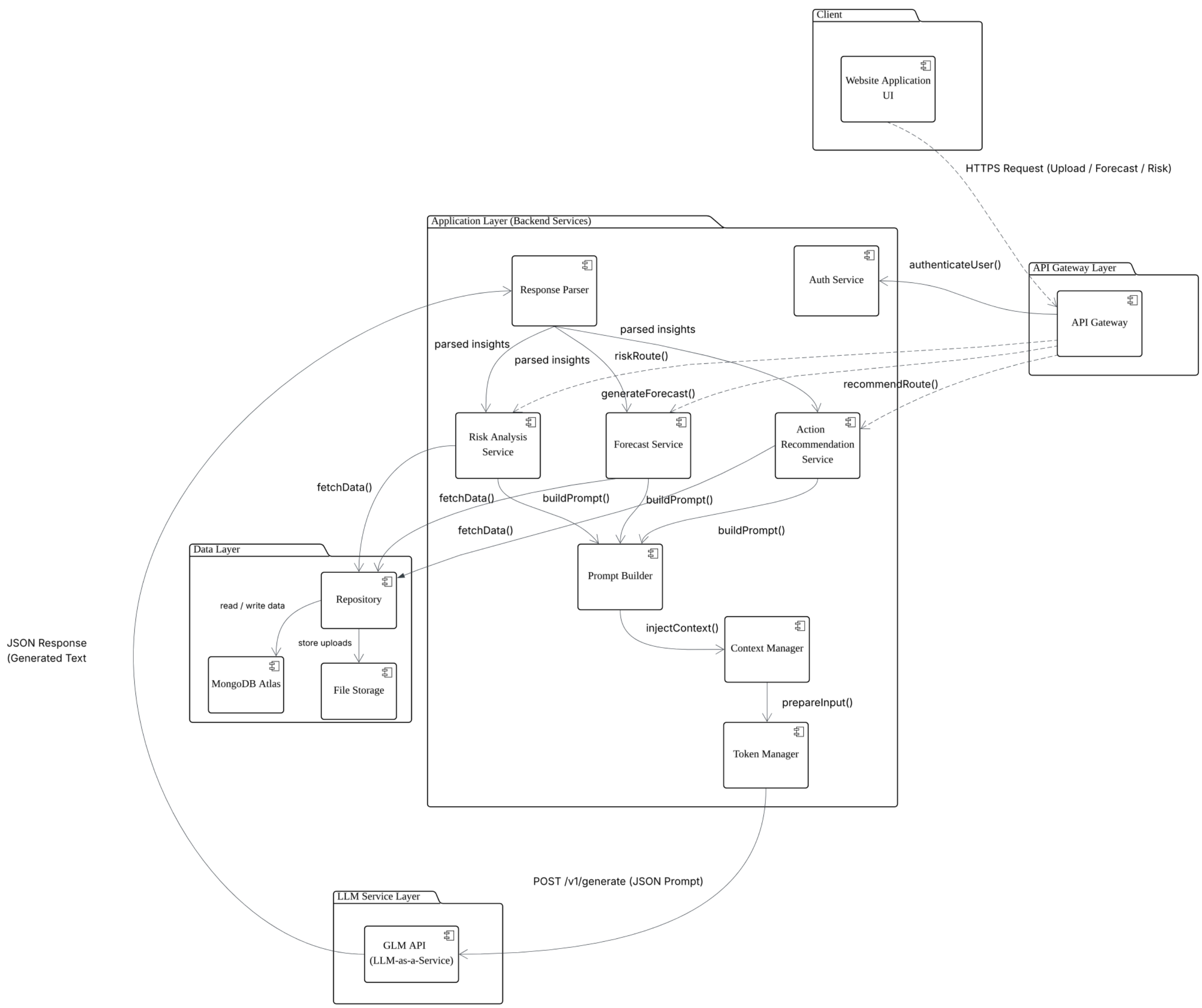
Type	Details
System	*Web Application (Flutter Web)*
Architecture	*Client-Server with Monolithic Backend and LLM-as-a-Service Layer*

FinSight AI is structured as a client-server system with a single-page Flutter web frontend communicating with a Node.js/Express backend via a RESTful API. The backend is organised into five feature-scoped service modules which is DocumentIngestionService, ForecastService, RiskAnalysisService, ActionRecommendationService, and a shared PromptBuilder/ContextManager/TokenManager pipeline. All five modules share a single MongoDB Atlas cluster accessed exclusively through a Repository class, which acts as the sole database gatekeeper. Binary file uploads are offloaded to Cloudflare R2 object storage. AI reasoning for Risk Detection, Cash Flow Forecast, and Recommendations is delegated externally to Z AI GLM via a POST /v1/generate HTTP call, making the LLM a stateless service layer rather than an embedded component.

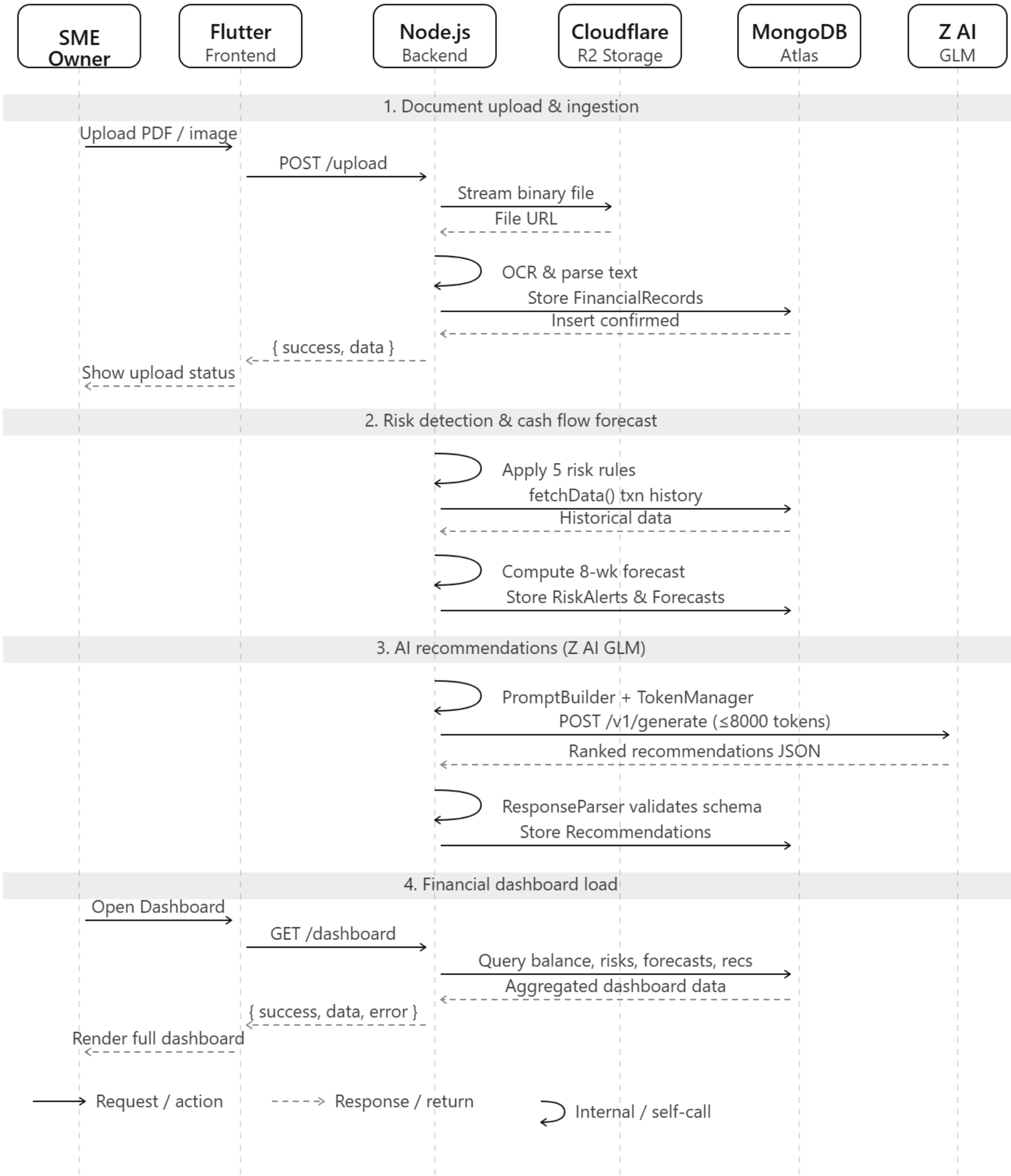
2.1.2 LLM as Service Layer

Z AI GLM is integrated as an external LLM-as-a-Service layer. When any of the three AI-dependent features (Risk Detection, Forecast, Recommendations) are triggered, the backend constructs a structured JSON prompt via the PromptBuilder, enriches it with business context through the ContextManager, and enforces a hard token ceiling of 8,000 tokens per call using the TokenManager — prioritising the highest-value overdue invoices and expense spikes through intelligent truncation before dispatch. The prompt is sent via POST /v1/generate to the Z AI GLM API. The returned JSON response is validated against a strict schema by the ResponseParser; any response failing validation falls back to the last successfully validated output, ensuring data integrity at all times.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



Email: umhackathon@um.edu.my

2.2. Technological Stack

FinSight AI is built on a carefully selected full-stack architecture that prioritises developer productivity, cross-platform reach, and seamless integration with the Z AI GLM LLM service. Each layer of the stack was chosen to directly support one or more of the five core features: Document Ingestion, Cash Flow Forecast, Risk Detection, AI Recommendations, and the Financial Dashboard.

2.2.1. Frontend - Flutter(Web)

FinSight AI is built on a carefully selected full-stack architecture that prioritises developer productivity, cross-platform reach, and seamless integration with the Z AI GLM LLM service. Each layer of the stack was chosen to directly support one or more of the five core features: Document Ingestion, Cash Flow Forecast, Risk Detection, AI Recommendations, and the Financial Dashboard.

2.2.2. Backend - Node.js (Express)

The application layer is a Node.js server built with Express, exposing a RESTful API contract of { success, data, error } across all five feature routes: /upload, /forecast, /risks, /recommendations, and /dashboard. Node.js was selected for its non-blocking I/O model, which is well-suited to FinSight AI's I/O-heavy workload: reading uploaded PDF and image files, making sequential HTTP calls to the Z AI GLM API, and reading and writing to MongoDB Atlas. The backend is organised into five feature-scoped service modules — DocumentIngestionService, ForecastService, RiskAnalysisService, ActionRecommendationService, and a shared PromptBuilder/ContextManager/TokenManager pipeline, keeping module boundaries clear for parallel development across the team. File uploads (PDF, JPG, PNG up to 15 MB) are handled by Multer middleware, which streams files to Cloudflare R2 object storage before returning a confirmation to the parser.

2.2.3. Database - MongoDB Atlas

Persistent data is stored in MongoDB Atlas, a fully managed cloud NoSQL database. MongoDB's document model is a natural fit for FinSight AI because financial records extracted from different document types (bank PDFs, invoices, receipts) arrive in varying schemas. Collections include Business, Transactions, Documents, DocumentChunks, FinancialRecords, Forecasts, ForecastScenarios, RiskAlerts, and Recommendations. Relationships between these collections are enforced at the application layer through the Repository class, which acts as the single gatekeeper for all reads and writes. The repository also implements soft-expiry logic: old AI-generated batches (risks, recommendations, forecasts) are replaced only with freshly validated records, ensuring the user is never shown stale or logically inconsistent financial advice.

2.2.4. Cloud/Deployment - Cloudflare R2

Binary file uploads (PDFs and receipt images) are stored in Cloudflare R2, an S3-compatible object store with zero egress fees. After a file is uploaded via the /upload endpoint, its binary data is streamed to R2 and the returned object URL is persisted in the Documents collection in MongoDB, linking the file reference to its parsed FinancialRecords. The OCR and document-parsing pipeline reads the file from R2 before passing extracted text to the AI engine.

2.2.5. LLM Integration — Z AI GLM (LLM-as-a-Service)

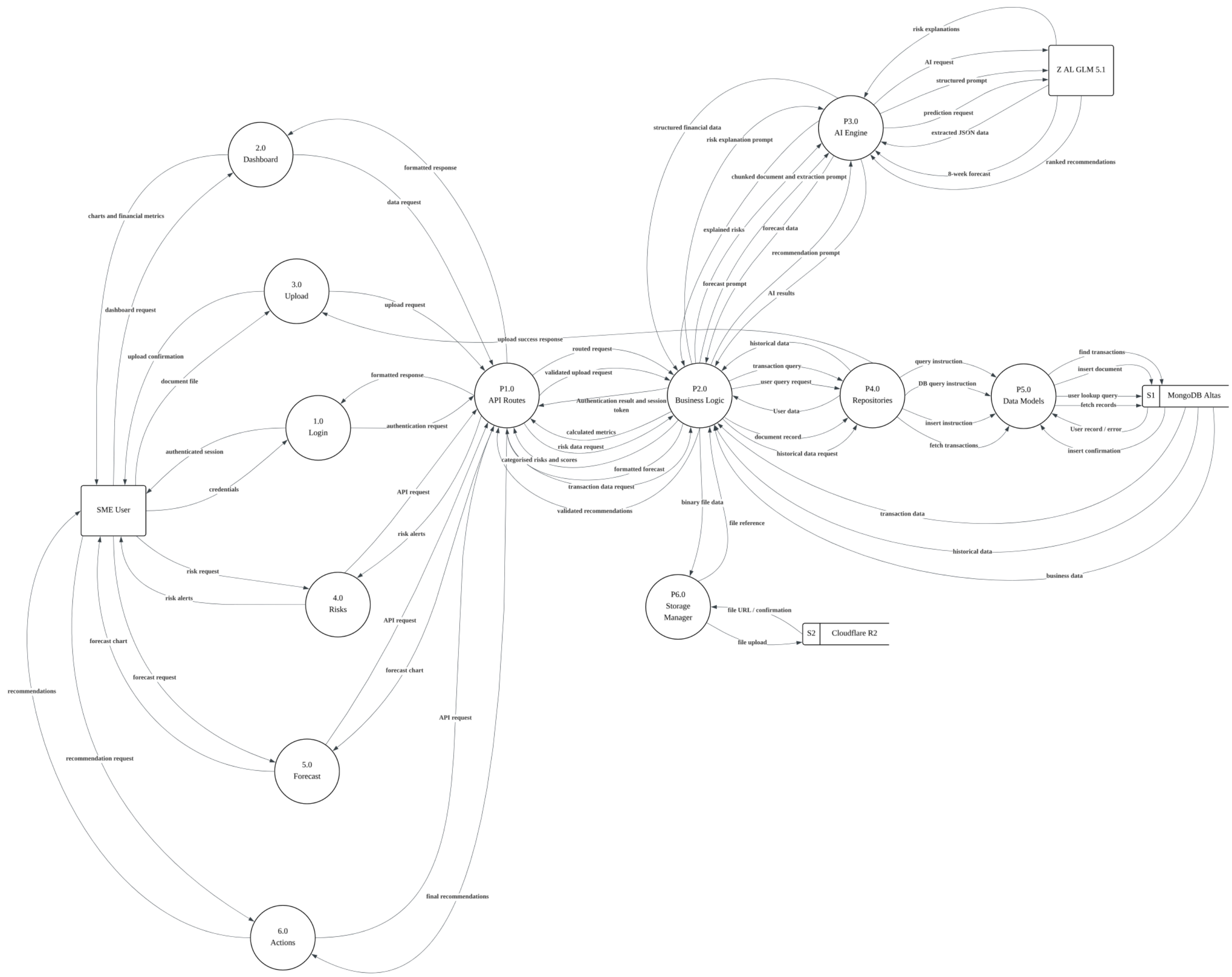
The AI reasoning layer is powered by Z AI GLM, integrated as an external LLM service via a POST /v1/generated HTTP API. The PromptBuilder constructs a structured JSON prompt from each service's financial snapshot (balance, overdue invoices, detected risks, upcoming expenses). The ContextManager injects relevant business context, and the TokenManager enforces a hard cap of 8,000 tokens per call by prioritising the highest-value overdue invoices and expense spikes through intelligent truncation before the request is dispatched. Responses are validated by the ResponseParser against a strict schema; any response that fails validation falls back to the last successfully validated output, maintaining data integrity at all times.

Technology Stack Summary

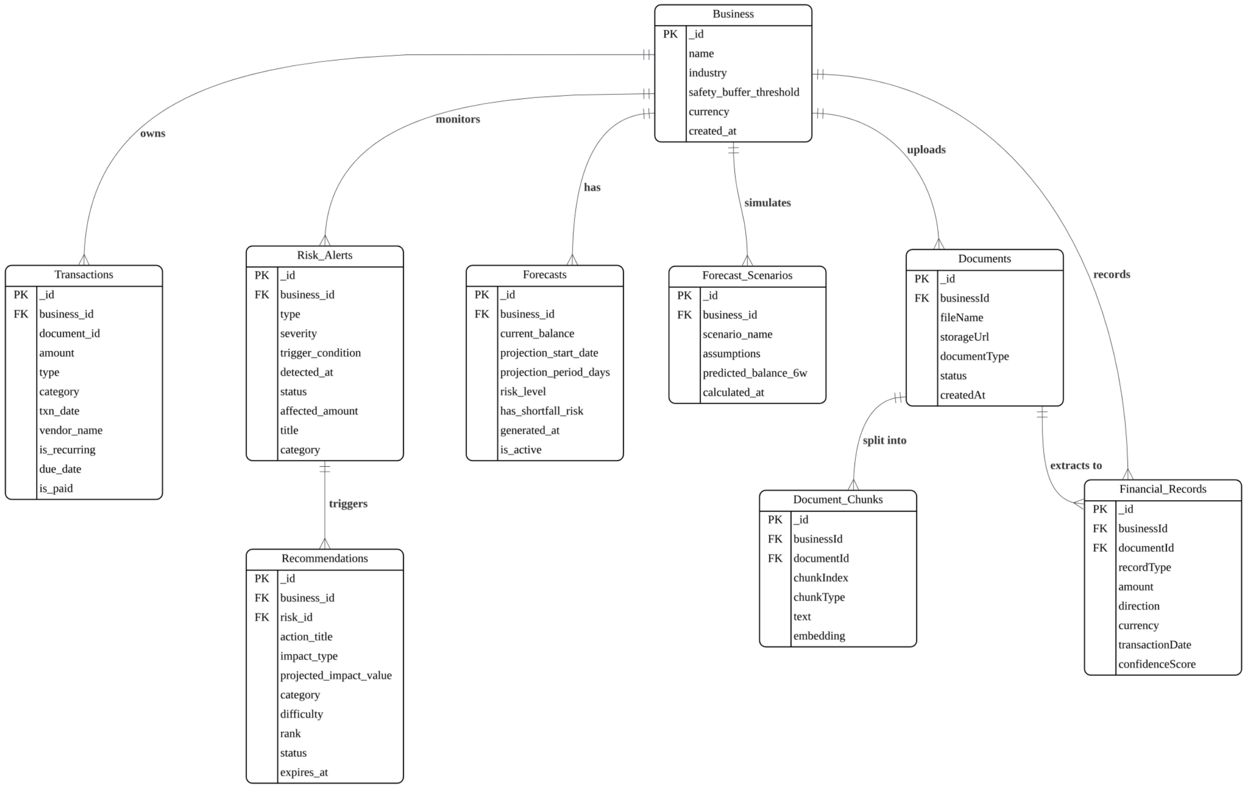
Layer	Technology	Justification	Feature Coverage
Frontend	Flutter (Web)	Single codebase, non-blocking async widgets, pixel-consistent UI	All 5 features
Backend	Node.js + Express	Non-blocking I/O, module-scoped services, simple REST routing	All 5 features
Database	MongoDB Atlas	Flexible document schema for varied financial record types	All 5 features
File Storage	Cloudflare R2	S3-compatible, zero egress fees, binary upload pipeline	Document Ingestion
LLM Service	Z AI GLM (API)	Instruction-following, context-aware reasoning for financial NLP	Risk, Forecast, Rec
Auth	JWT + API Gateway	Stateless token auth, route-level gating	All 5 features

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

#	*Features*	*Description*
1	*Document Ingestion & Parsing*	*SME owners upload PDFs (bank statements, invoices) or JPG/PNG receipts up to 15 MB via the Upload Documents screen. The AI parser (Ilmu AI) extracts line items and structures them into transaction records. Previously uploaded files are listed below the upload area with their filename, file size, page count, parser type, upload date, and ingestion status (uploaded / completed).*
2	*Cash Flow Forecast*	*8-week projection of weekly inflow, outflow, net and projected balance, derived from historical transaction patterns. Each week is labelled SAFE or CASH GAP in colour-coded status badges. A bar + line visualisation shows inflow/outflow bars alongside the projected balance trend line. A summary banner below the chart flags the specific*

		cash-gap week with the projected negative balance amount, and a bottom prompt quantifies the RM impact of following the top recommendations.
3	*Risk Detection & Alerts*	*Five deterministic rules, namely cash_flow_gap, overdue_invoices, expense_spike, revenue_decline, and large_payable, are applied to the transaction history. Each detected risk is assigned a severity level, which can be Critical, High, Medium, or Low, together with an affected_amount in MYR, a timing indicator such as "Now" or "Week 6 from now", and a plain-English explanation generated by Ilmu AI. An overall Financial Risk Score ranging from 0 to 10 is calculated based on the number and severity of detected risks, and a next review date is also provided.*
4	*AI Decision Recommendations*	*Ilmu AI consumes a financial snapshot (balance, overdue invoices, upcoming expenses, cost areas) and returns ranked actions across four categories (Collections, Supplier Management, Cost Optimization, Financing). Each action has a difficulty rating, timeframe, projected RM impact, 3–5 concrete steps and, where relevant, an invoice reference. Users can expand individual recommendations to read the AI reasoning and mark actions as completed via "Mark as Actioned."*
5	*Financial Dashboard*	*Single-screen overview combining: Current Balance, Monthly Revenue, Monthly Expenses, Outstanding Invoices (with month-on-month percentage change badges); a Cash Balance Trend chart showing historical data plus the 8-week forecast; a live Risk Alerts panel; a live AI Recommendations panel (top 3 actions by impact); and a Recent Transactions feed. A Critical Alert banner at the top of the page surfaces the most urgent risk with a direct "View Risks" call-to-action button. A persistent bottom bar shows the count of urgent risks and the last-updated timestamp.*

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	Handle concurrent uploads and AI requests from multiple SME users without performance degradation.	Backend coding scaled via Node.js distributes incoming traffic. It queues Ilmu AI

		requests to prevent API overload.
Reliability	99.5% uptime for document ingestion and dashboard. Failed parses must never corrupt existing transaction records.	Parsing jobs tracked with statuses. On failure, the job rolls back like no partial writes reach the transaction table. Dashboard serves cached data if AI is temporarily down.
Maintainability	All five feature modules must be independently deployable without affecting one another.	Each module follows a consistent API contract with the repository layer as the sole DB gatekeeper. Separate CI/CD pipelines per module on GitHub Actions.
Token Latency	Ilmu AI calls must complete within 3 minutes, capped at 8000 max tokens. Flutter UI must not block while awaiting responses.	Flutter renders deterministic data immediately while AI panels show the loading of the generation button. After successful generation, the results will be displayed via ranking and further details.
Cost Efficiency	Token usage per session must stay within budget. Identical financial snapshots must not trigger repeated API calls.	Prompts templated and kept under 8000 tokens. Documents chunked before parsing.

3.3. Out of Scope / Future Enhancements

FinSight AI focuses on delivering a high-impact MVP within the hackathon timeline. The following features are part of the long-term vision but are not included in the current implementation:

a. ***Real-time Bank API Integration***
   Direct integration with Malaysian banking APIs (e.g., Maybank, CIMB) to automatically sync transactions instead of manual document uploads.
b. ***Conversational AI Financial Assistant (Chat Interface)***
   An in-app chat feature allowing SME owners to ask financial questions (e.g., "Can I afford to pay supplier X this week?") and receive contextual responses.
c. ***Multi-Business Account Management***

   Support for users managing multiple businesses under one account, with separate dashboards and financial tracking.

d. ***Advanced Predictive Analytics (Beyond 8 Weeks)***

Extending forecasting from 8 weeks to long-term projections using more complex predictive models and seasonal trend analysis.

e. Automated Expense Categorization & Tax Reporting

AI-powered categorization of expenses into tax-relevant categories and generation of tax-ready reports.

f. Mobile Application (Flutter Mobile Deployment)

The current system is web-based; a dedicated mobile app version is planned for better accessibility.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing

FinSight AI adopts a progressive grayscale rollout strategy for any new feature releases following the initial hackathon deployment. New capabilities — such as an updated risk scoring algorithm or a revised recommendation prompt template — are first released to 5% of active SME users while the remaining 95% continue on the stable version. System behaviour is monitored over a 24-hour observation window, tracking key metrics including AI output pass rate, API response latency, and error rate. If no anomalies are detected and all CI/CD thresholds are met, the rollout is incrementally expanded to 25%, 50%, and finally 100% of users.

Once a feature reaches full rollout, A/B testing is conducted where applicable to determine which variant performs better. For example, two prompt templates for the AI Recommendations feature may be tested against each other — Variant A using a multi-step structured prompt and Variant B using a condensed single-step prompt — with acceptance measured by AI output pass rate, recommendation relevance score, and average token consumption per session. The better-performing variant is then adopted as the new default.

4.1.2. Emergency Rollback (ER) & Golden Release

FinSight AI maintains a designated Golden Release — the last fully verified and stable build that has passed all CI/CD thresholds including 100% unit test pass rate, zero P0/P1 bugs, minimum 90% regression test pass rate, and API response time below 800ms. This Golden Release is tagged in the GitHub repository and its Docker image is retained in the container registry.

If any of the following anomalies are detected post-deployment, an Emergency Rollback is triggered automatically via the GitHub Actions pipeline:

- P0 or P1 bug detected in production (e.g. AI output consistently failing schema validation, dashboard returning empty data for all users)

- API error rate exceeding 5% over a 10-minute window
- Average response time exceeding 3 minutes for Z AI GLM-dependent endpoints
- MongoDB Atlas write failure rate exceeding 1%

Upon trigger, the automated pipeline redeploys the Golden Release image to the staging and production environments within approximately 5 minutes, restoring service. A Slack notification is dispatched to all team members with the rollback reason, the anomaly metric that triggered it, and the commit hash of the failed deployment. A post-mortem review is conducted before any subsequent deployment attempt.

4.1.3. Priority Matrix:

Priority Level	Trigger Condition	Action
P0 - Critical	AI output schema validation failure rate exceeds 20% in a 5-minute window; MongoDB Atlas becomes unreachable; dashboard returns 500 errors for all users	Immediate Emergency Rollback to Golden Release. All team members were notified. No new deployments until root cause is resolved and a fix is verified in staging.
P1 - High	Z AI GLM API response time consistently exceeds 3 minutes; Cloudflare R2 upload failure rate exceeds 5%; unit test pass rate drops below 100% on main branch	Immediate Emergency Rollback to Golden Release. All team members were notified. No new deployments until root cause is resolved and a fix is verified in staging.
P2 - Medium	AI output pass rate falls below 80% but above 60%; forecast arithmetic deviation exceeds 0.01 MYR tolerance; code coverage drops below 81.2%	No rollback. Hotfix branch created and reviewed within 4 hours. Deployed via standard CI/CD pipeline after passing all thresholds.
P3 - Low	ESLint linting errors introduced on a feature branch; non-critical UI rendering inconsistency; stale recommendation data displayed with correct timestamp	Logged in defect tracker. Resolved in the next scheduled sprint cycle. No immediate deployment action required.

4.2. *Monitoring*:

FinSight AI's monitoring plan covers four layers of the system: infrastructure, application, AI service, and data integrity, with defined alert thresholds and escalation actions for each.

Infrastructure Monitoring The Node.js backend process is monitored via AWS CloudWatch, tracking CPU utilisation, memory consumption, and request throughput on the ECS container. If CPU utilisation exceeds 80% for more than 3 consecutive minutes, an Auto Scaling Group scale-out event is triggered, adding a new ECS task instance. The Application Load Balancer health check pings the /health endpoint every 30 seconds; if two consecutive checks fail, the unhealthy instance is deregistered and replaced automatically.

Application-Level Monitoring All five API routes (/upload, /forecast, /risks, /recommendations, /dashboard) emit structured logs to AWS CloudWatch Logs on every request, recording the endpoint, HTTP status code, response time in milliseconds, and business_id. A CloudWatch Alarm is configured to fire if the 5-minute rolling error rate on any route exceeds 5%, notifying the backend developer via email and triggering a P1 escalation review.

AI Service Monitoring Every call to Z AI GLM is logged with the prompt token count, response token count, response time, and ResponseParser validation outcome (pass or fail). A daily summary report is generated tracking the AI output pass rate across all three AI-dependent features. If the pass rate falls below 80% on any given day, a P2 alert is raised and the prompt templates for the affected feature are reviewed. Token usage per session is tracked against the 8,000-token budget; sessions consistently approaching the cap trigger a review of the TokenManager truncation logic.

Data Integrity Monitoring The Repository class logs every read and write operation to MongoDB Atlas, including collection name, operation type, document count, and execution time. If any write operation fails, the error is logged and surfaced as a P1 alert. The soft-expiry mechanism — which replaces old AI-generated batches only with freshly validated records — is verified on every successful AI generation cycle. A weekly automated script checks for orphaned DocumentChunks (chunks with no associated Document record) and orphaned FinancialRecords (records with no associated Business document), reporting any found to the data validation specialist.

4.3. Assumptions:

The following key operational and environmental conditions were assumed to be valid during the development and deployment of FinSight AI:

a. SME owners have a stable internet connection on their device when accessing the Flutter web application, uploading documents, and viewing the dashboard. The system does not support offline mode or background sync.

b. Uploaded financial documents are legible and in a supported format (PDF, JPG, or PNG, up to 15 MB per file). The OCR and document parsing pipeline assumes that text in uploaded bank statements and invoices is machine-readable or sufficiently clear for OCR extraction. Handwritten documents may produce degraded extraction quality.

c. Z AI GLM API is available and reachable during all AI-dependent operations. The system assumes a maximum response time of 3 minutes per call. In the event of Z AI GLM unavailability, the graceful degradation fallback — serving the last validated AI output with a stale-data timestamp — is assumed to be an acceptable temporary user experience.

d. MongoDB Atlas and Cloudflare R2 are available with their standard managed service SLAs. The system does not implement its own database replication or object storage redundancy beyond what these managed services provide natively.

e. Each SME user session is assumed to consume approximately 8,000 tokens across all Z AI GLM API calls. Users who upload unusually large document sets or trigger all five AI-dependent features in rapid succession may approach or exceed this per-session estimate, which is managed by the TokenManager truncation logic.

f. The synthetic financial datasets used during development and QA — representing a café and bakery, an auto repair workshop, and a pet grooming operator — are assumed to be sufficiently representative of the transaction patterns of the target SME user segments for the purpose of validating the risk detection rules and forecast arithmetic during the hackathon sprint.

g. Users are assumed to be operating the system on a modern desktop or laptop browser (Chrome, Firefox, or Edge). The Flutter web application is not optimised for mobile browsers and is not delivered as a native mobile application in this version.

4.4. ***External Dependencies:***

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*
Google Maps API	*Tracking rider location and optimize the route*	*Medium level- if real time tracking is being disabled.*

5. ***Project Management & Team Contributions***

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. ***Project Timeline:***

Phase	Duration	Activities
Planning and Setup	Day 1 - 2	Problem scoping, SME niche definition, synthetic dataset preparation, role assignments, repo setup

		and API environment configuration for Z AI GLM.
Core Development	Day 3 - 5	Parallel feature development begins. Document ingestion and parsing (Wanye), Cash Flow Forecast (ChernTak), Risk Detection and Alerts (Zhi Hui), AI Recommendations with Z.AI (Xin Yi) and Financial Dashboard (Chillien).
Integration	Day 6 - 7	Backend modules connected to Flutter frontend. API contract { success, data, error } enforced across all five features. AI output wired into Dashboard panels. End-to-end flow tested.
Validation and QA	Day 8 - 9	Scenario-based testing across happy path and negative path cases. AI arithmetic validation, token usage checks, NFR benchmarking (latency, fallback behaviour). Bug fixes.
Finalisation	Day 10	Demo preparation, pitch video recording, documentation cleanup, final deployment to staging environment.

5.2. ***Team Members Role:***

Team Member	Role	Responsibilities	Feature Ownership
Lew Zhi Hui	Product Owner / Business Analyst	Defines the target SME niche and quantifiable impact. Responsible for sourcing or generating realistic financial datasets, aligning all features to a real business problem and leading to the final pitch and demo video.	Risk Detection and Alerts
Lee Xin Yi	AI / Prompt Engineer	Owns the Z AI GLM integration end-to-end. Handles prompt chaining and the Decision Intelligence logic behind AI recommendations.	AI Decision Recommendations (Z.AI as engine)
ChernTak	Backend Developer	Sets up the backend of the system and API, manages file upload handling (PDF/Images) and connects all AI outputs to the Flutter frontend reliably.	Cash Flow Forecast
Wanye	Frontend / UX Developer	Builds up the Figma Mock-up and Flutter UI across the overall system. Focuses on clarity like traffic-light cash flow indicators and designs test	Document Ingestion and Parsing

		scenarios.	
Chillien	Data and Validation Specialist	Validates AI arithmetic outputs (GLMs can miscalculate raw figures) and quantifies the measurable impact of the solution (time saved, cost reduced) for the final presentation.	Financial Dashboard

5.3. *Recommendations:*

- Scalability: Introduce a lightweight job queue between the file upload endpoint and the Ilmu AI parser in app.js to handle concurrent uploads without blocking the main thread with a future path to split the backend into feature-scoped modules as the user base grows.
- Performance: Cache Z.AI outputs in Redis for 24 hours and pre-compute the Cash Flow Forecast on each successful document ingestion, eliminating redundant API calls and unnecessary recalculations on every dashboard load.
- Reliability: Implement a graceful degradation state on all Z.AI dependent Flutter panels (showing the last successful output with a stale-data timestamp on timeout), backend by an emergency rollback pipeline on GitHub Actions for rapid recovery from any breaking deployment.